



SERVIÇO PÚBLICO FEDERAL · MINISTÉRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE VIÇOSA · UFV
CAMPUS FLORESTAL

Trabalho Prático - Parte 3: Caldeirão

Autores (as):

Sabrina Bruni de Souza Faria [EF 05081]

Luiz Guilherme de Sá Gontijo [EF 04839]

Breno Júnio de Oliveira Gomes [EF 05085]

Disciplina: Sistemas Distribuídos e Paralelos [CCF - 355]

Sumário

1. Introdução 🌐	3
2. Negócio 🏠	3
3. Processos ⚙️	3
3.1. Cliente 🧑	4
3.2. Servidor de Aplicação 🖋️	5
3.2.1 Exemplo de operação ✉️	6
3.3. Servidor de Banco de Dados 📦	7
4. Modelagem 📐	8
4.1. Diagrama de Classes 📄	8
4.2. Diagrama de Banco de Dados 🗄️	8
4.3. Diagrama de Threads 🧵	9
4.4. Comunicação 📡	11
4.4.1. Troca de Mensagens ✉️	11
4.4.2. Representação externa de dados 📄	12
4.4.3. Envio e recebimento 📧	13
5. Resultados 📊	13
6. Dificuldades 🧱	13
7. Conclusão 📋	14
8. Referências 📖	14

1. Introdução 🌟:

Este trabalho tem como objetivo apresentar a implementação de um Sistema Distribuído construído por meio de comunicação entre processos utilizando API de Sockets. O SD implementado é um sistema de marketplace chamado Caldeirão, um sistema de compras e vendas de poções mágicas.

2. Negócio 🏰:

Nesse sistema, os usuários compram e vendem poções mágicas. Dessa forma, um novo usuário no sistema pode somente visualizar os anúncios de produtos e lojas, e pesquisar por algum anúncio com base no nome do produto. Para efetivamente realizar uma compra, o usuário deve realizar um cadastramento ou um login.

Após se identificar, o usuário além de poder realizar uma compra, ele pode criar lojas para vender suas poções. Nessas lojas, ele pode criar produtos nos quais ficarão visíveis somente para ele; criar anúncios desses produtos para divulgar para outros usuários, definindo um preço e quantidade disponível; e visualizar os pedidos que foram feitos na loja, confirmando ou cancelando-os.

O usuário também pode editar seus dados de cadastramento, adicionar e editar endereços e visualizar suas compras realizadas. Os dados referentes a uma loja, um anúncio e um produto também podem ser editados. Todas as opções de edição incluem exclusão, menos no caso de editar dados do usuário.

Ao criar uma loja, tem-se a opção de adicionar ou não uma imagem. No caso de produto, é exigido pelo menos uma imagem. Ao criar um anúncio, deve-se adicionar a chave pix a qual deve ser usada para os pagamentos.

Ao realizar uma compra o usuário precisa escolher um endereço e a quantidade desejada do produto. Após isso, ele deve realizar o pagamento, que no caso é somente via pix, tendo a opção de QR Code e Cópia e Cola. O usuário confirma que fez o pagamento e o vendedor deve confirmar ou cancelar esse pedido.

3. Processos ⚙️:

O SD foi dividido em três processos: cliente, servidor de aplicação e servidor de banco de dados. Essa escolha de implementação foi para deixar mais bem separada a parte da aplicação do crúde dos dados do sistema.

Sendo assim, o processo cliente faz requisições ao processo servidor de aplicação que faz requisições ao processo servidor de banco de dados. O servidor BD responde o de aplicação que responde ao cliente. Em alguns casos a troca de mensagens entre dois processos ocorre mais de uma vez antes de se comunicar com o terceiro.

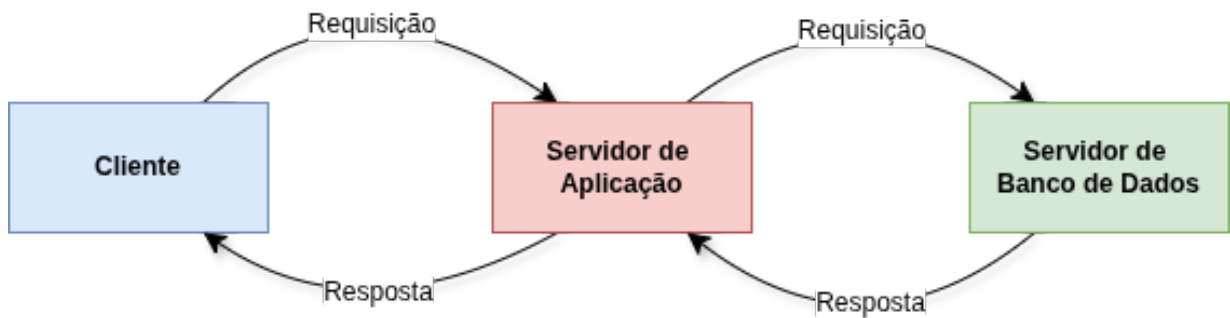


Figura 1: Comunicação entre processos

3.1. Cliente 🧙:

O processo cliente é dividido em três partes: interface, aplicação e socket. A interface são as telas que o usuário interage, acompanhadas de funções que recebem e realizam algo a depender da resposta da aplicação. A aplicação é a parte composta por definição das mensagens a serem enviadas e recebidas para cada tipo de requisição. O socket em si é o procedimento de enviar a mensagem montada na aplicação ou receber uma mensagem e passar para a aplicação.

O cliente é responsável por processar as requisições do usuário via interface, processar a função referente a requisição na aplicação e passar para o socket fazer a troca de mensagens. Dessa forma, o cliente recebe respostas do servidor de aplicação pelo socket, passa a resposta para a aplicação que responde a interface que decide o que deve ser feito na tela.

Para construir a interface foi utilizado o framework **PyQt6**, como exigido na especificação. A interface em si foi dividida em duas classes: a classe principal responsável por criar e armazenar cada uma das telas, e uma classe responsável por fazer a interação na tela de acordo com a resposta de uma requisição pela aplicação do cliente.

Além disso, foram criadas outras classes auxiliares à interface, como: classes para criar formulários; classe com funções gerais para criar um botão, uma imagem, etc; classe com manipulação de telas; e classe de validação de dados e geração de código pix. Para gerar o código pix foi utilizada a biblioteca **pybrcode** que foi incluída manualmente no repositório no sistema.

Para garantir que a interface não fique travada ao esperar uma requisição ser retornada pelo servidor, foi utilizada a biblioteca **QThread** do próprio framework **PyQt6**. Dessa forma, foram criadas funções em uma classe para realizar a manipulação das requisições em threads de modo que mostre uma tela de carregamento enquanto a requisição é processada. Existem duas ações diferentes para serem realizadas quando a requisição termina: abrir uma nova tela ou mostrar somente uma mensagem na tela antes do carregamento.

Outro ponto importante é que foi adicionado um timeout de 15 segundos para caso a requisição nunca tenha sido enviada e/ou a resposta nunca chegue. Sendo assim, quando o timeout acontece é exibida uma mensagem de erro, mas que não

impede o usuário de continuar navegando no sistema, permitindo que ele faça outras requisições.

3.2. Servidor de Aplicação :

O servidor é responsável por receber e processar as requisições da aplicação cliente, e de repassar as respostas do banco de dados. Este processo é melhor descrito dividindo-o em três partes principais: Client Handler, Fila de Mensagens, Callback. O Client Handler é a parte responsável por receber requisições de clientes, entender o tipo de operação, e preparar a mensagem correspondente para o servidor de banco de dados. A Fila de Mensagens corresponde à parte indireta da comunicação neste sistema. Ela atendeu ao objetivo principal que era o de evitar sobrecarga no servidor de banco de dados. A implementação dessa fila de mensagens no nosso sistema não se deu por meio de outro processo, mas por uma estrutura dentro do servidor, rodando em outra thread. Por fim, a parte de Callback é o retorno para o cliente, a resposta à sua requisição.

O Client Handler compreende todo o momento, desde quando o servidor recebe a requisição do cliente até a requisição chegar à fila de mensagens. Quando um cliente se conecta ao servidor, uma nova thread se inicia para lidar com as requisições deste cliente. A mensagem é recebida, decodificada e, de acordo com o tipo de operação requisitada, levada para outra thread que processa essa mensagem e prepara: a mensagem do servidor para o banco, e a forma de tratar o retorno para o cliente. Essas informações são enfileiradas na fila de mensagens e, quando o servidor de banco de dados estiver livre, são processadas e geram a resposta adequada para o cliente. Todas as mensagens que o cliente manda para o servidor são tratadas aqui. As requisições normais, a continuação do cadastramento (o código de confirmação), e as imagens que o cliente manda em operações de criação de imagens ou de itens com imagens. Toda a recepção de mensagens ocorre na classe ClientHandler do código, o processamento das mensagens acontece em classes auxiliares correspondentes à operação em questão, esse processamento ocorre em uma thread diferente, assim, o servidor pode lidar com diferentes requisições de um cliente se for o caso -o banco de dados vai lidar com uma de cada vez, sem pressão, graças à fila.

A Fila de Mensagens é uma fila que se inicia junto ao servidor. A classe Fila de Mensagens do código representa essa entidade. Em nosso sistema, ela tem duas funções superimportantes: manter a mensagem do servidor e levá-la para o banco de dados quando este último estiver livre; e manter os dados de um usuário que está em processo de cadastramento. Quando o banco está livre, a fila desenfileira os dados preparados na etapa anterior, envia a(s) mensagem(ns) do servidor de aplicação para o banco de dados, recebe a resposta deste e redireciona essa resposta para o callback adequado para o fim dessa conversa com o cliente. A fila tem uma fila da biblioteca **Queue** que implementa thread locks automaticamente, evitando conflito entre requisições de diferentes clientes. Além dessa fila principal de requisições, a fila possui outro campo que mantém dados de usuários que estão

fazendo a operação de cadastramento -uma operação que envolve uma conversa inicial com o usuário que requisita seu cadastro, e uma conversa posterior quando o usuário envia o código de confirmação recebido por email. Nesse caso, a segunda parte da operação de cadastramento não conversa com o banco de dados, os dados ficam no servidor de aplicação e a operação é considerada cancelada depois de três minutos e o cliente deve requisitar um novo cadastro.

O Callback é a parte final da conversa. As funções de callback são as responsáveis por entregar a devida resposta ao cliente com base na mensagem do servidor de banco de dados. São as funções que retornam os dados do banco e a mensagem que o cliente espera para sua aplicação tomar suas decisões internas.

3.2.1 Exemplo de operação :

Para explicitar o funcionamento do servidor de aplicação, vamos detalhar um pouco o fluxo de uma das operações do sistema pelo servidor: o cadastramento.

O Servidor recebe uma mensagem do cliente. Ele decodifica a mensagem e confere o tipo de operação em questão. Identificada a operação de cadastramento, o servidor prepara uma requisição para o Banco de Dados conferir os dados que o cliente mandou, separa o callback para essa operação de cadastramento, e enfileira essas informações na fila de mensagens.

A fila tem, agora, esses dados. À disponibilidade do banco de dados, os dados são desenfileirados, e a requisição do servidor é enviada para o banco. O banco retorna uma mensagem ao servidor, informando o status da tentativa de cadastramento. Essa mensagem é recebida e direcionada à função callback correspondente.

O servidor processa a mensagem do banco de dados e elabora a resposta para o cliente, que, nesse caso, é avisá-lo que um código de confirmação foi enviado ao email fornecido -nesse caso vamos assumir que o usuário inseriu dados válidos e pode ser cadastrado. O servidor então guarda os dados do cliente junto ao código de confirmação.

Para o envio de email via Sockets foi necessário utilizar o protocolo SMTP (Simple Mail Transfer Protocol) por meio da biblioteca **smtplib**. Essa biblioteca não usa HTTP(S) e não depende da web no sentido tradicional (como usar um navegador ou chamadas REST). Ela se comunica diretamente com o servidor SMTP via protocolo TCP, e SMTP sobre SSL/TLS como uma camada de segurança.

Posteriormente, o cliente envia um código. Este é processado e comparado com o código que estava guardado. Os dados são acessados pelo socket do cliente, então, se ele fechar a conexão, terá de tentar realizar o cadastramento novamente. Se os códigos baterem, o servidor enfileira uma requisição para o banco de dados cadastrar o cliente no sistema. O caminho a partir daqui é semelhante ao primeiro passo de cadastramento, ao passo que, no fim, o cadastramento propriamente dito será efetuado.

3.3. Servidor de Banco de Dados :

O banco de dados SQLite é criado importando `caldeirao.sql`, o conjunto de instruções que cria o banco. Na mesma pasta que o banco, ficam as pastas onde serão armazenadas as imagens das lojas e dos produtos. O servidor de banco de dados consiste essencialmente no arquivo `servidor-bd.py`. Este arquivo executa um processo que, através de inúmeras Classes e estruturas de dados, ouve por conexões, recebe requisições de outras máquinas e manipula o banco de dados conforme for solicitado. Seguindo o padrão estabelecido de Troca de Mensagens, entende-se que o servidor de aplicação quer do servidor de banco, que executa a(s) ação(ões) desejada(s) e retorna mensagens, geralmente mostrando como ficaram os dados no banco de dados.

Quando o servidor recebe uma requisição, ele escuta a primeira mensagem, sempre esperando uma string, e, a partir do que houver nela, monta uma resposta. Se a resposta montada esperar imagens, é preparada uma lista de caminhos, nos quais serão salvas imagens, enviadas posteriormente, uma a uma. Há somente dois casos em que uma imagem é armazenada: Loja e Imagem_Produto. Como o servidor não poderia manter nomes customizados, achamos mais prudente deixar um padrão `{id_loja}.jpg` para as imagens de uma loja e `{id_imagem_produto}_{id_produto}.jpg` para a imagens de produtos. Assim, se pouparia, no banco de dados, de armazenar a string de um deles, já que assim, o nome da loja e do produto passam a ser atributos relativos, podendo ser calculados a partir dos outros atributos.

4. Modelagem

4.1. Diagrama de Classes

O sistema foi modelado com orientação a objetos, de modo que cada componente importante de dados seja uma classe única que utiliza ou possui outras classes. Com base na modelagem de casos de uso do sistema, as classes foram modeladas da seguinte forma:

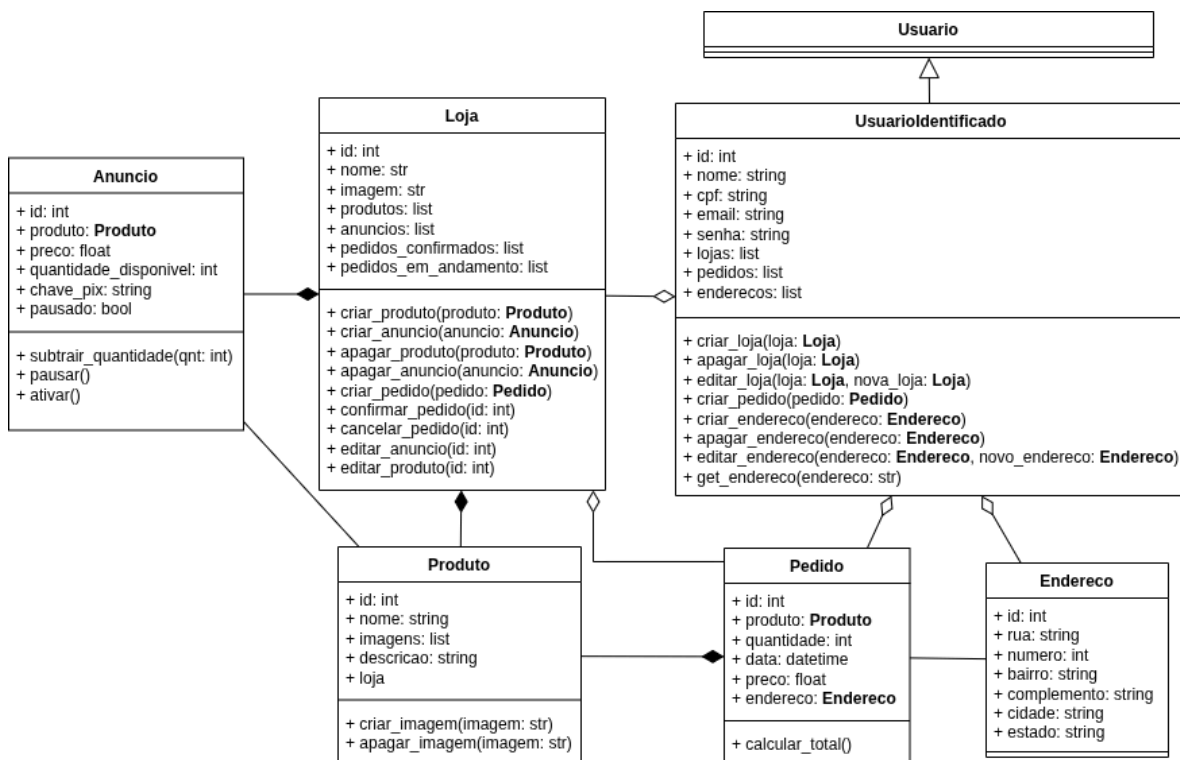


Figura 2: Diagrama de Classes

4.2. Diagrama de Banco de Dados

O banco de dados foi modelado com base no diagrama de classes, mas sem redundância de dados. Dessa forma, o banco de dados foi modelado da seguinte forma:

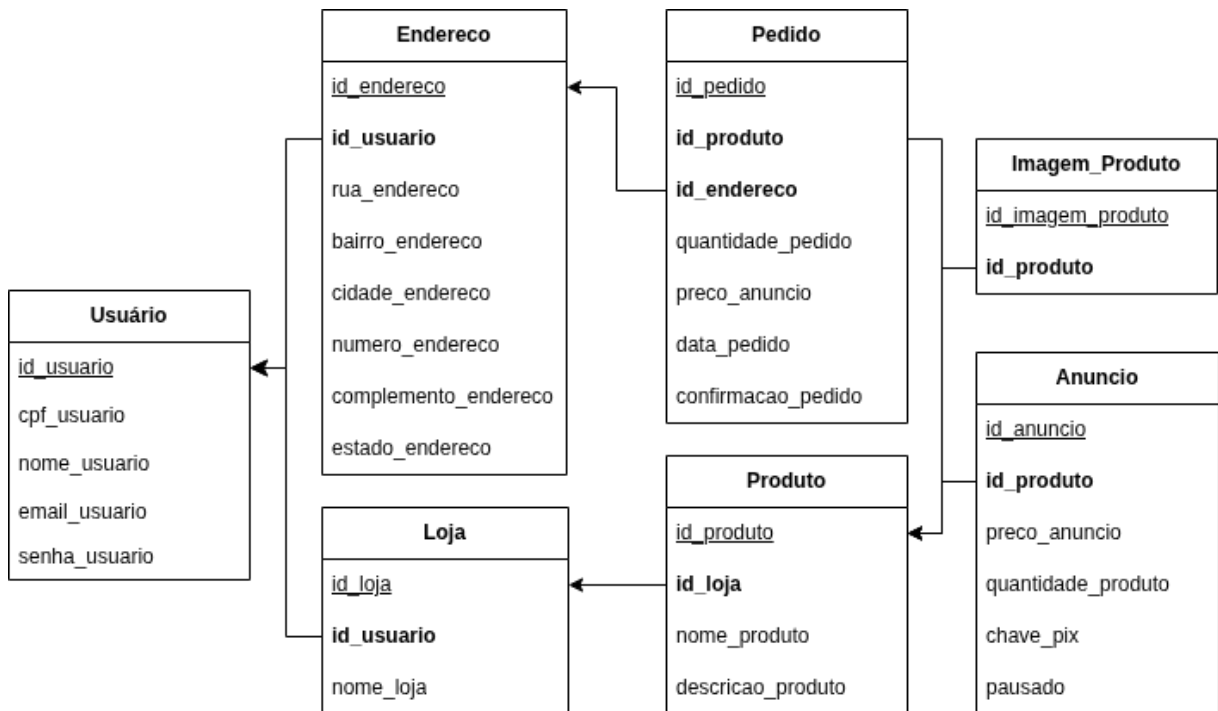


Figura 3: Diagrama de Banco de Dados

4.3. Diagrama de Threads 🧵 :

Foram usadas threads na interface, mas de forma muito sucinta, somente para mostrar uma tela de carregamento enquanto a resposta não chega ou mostrar uma mensagem de erro caso dê timeout, como explicado anteriormente.

No servidor de aplicação, as threads foram usadas de forma muito mais intensa, para que fosse possível processar requisições de inúmeros clientes ao mesmo tempo. Sendo assim, cada tipo de requisição é uma thread que manda para o banco de dados um padrão de requisição. No servidor de banco de dados não foram utilizadas threads, sendo as requisições processadas sequencialmente.

Para melhor visualizar e planejar a interação entre threads e requisições para o banco foi inventado e criado um diagrama de threads que representa a interação das threads dentro do servidor de aplicação. Esse diagrama segue a seguinte lógica:

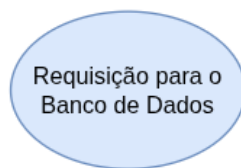
- Para representar uma entidade fora do servidor:



- Para representar uma thread:



- Para representar uma requisição do servidor de aplicação para o servidor de banco de dados:



- Para representar o fluxo entre os componentes:



A thread **Read** é a thread que contém um switch case entre as opções de requisição. Com isso, o diagrama de threads ficou da seguinte forma:



Figura 4: Diagrama de Threads

Esse diagrama pode ser visualizado melhor entre os arquivos de artefatos.

4.4. Comunicação .

4.4.1. Troca de Mensagens .

A comunicação entre os processos foi definida seguindo um padrão de separação em tópicos por meio do caractere “|” (pipe). O padrão definido é uma palavra que define a requisição seguida de pipe e dados utilizados na requisição.

Em alguns casos existe mais de um tópico para especificar a requisição, e em outros casos existe o envio de imagem após mensagem. Esse padrão é utilizado tanto para a comunicação entre cliente e servidor de aplicação quanto para a comunicação entre servidores.

Para melhor explicar as trocas de mensagens, junto aos artefatos está um documento de troca de mensagens, detalhando cada tipo de requisição entre todos os processos. Seguindo a seguinte legenda:

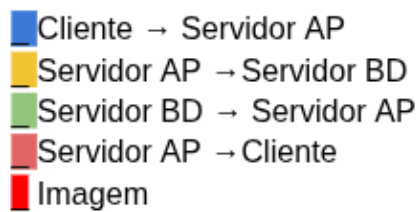


Figura 5: Legenda da troca de mensagens

Um exemplo de troca de mensagens é para o caso de realizar login. O Cliente envia para o servidor de aplicação uma mensagem dizendo que deseja realizar um login e os dados deste login. O servidor recebe esses dados e envia para o servidor de banco de dados uma requisição de login com os dados. O servidor BD vai analisar se os dados estão corretos e retornar duas possibilidades: ok com os dados simples do usuário completos, caso os o login tenha sido bem sucedido; ou erro indicando dados incorretos, caso os dados de login não sejam válidos.

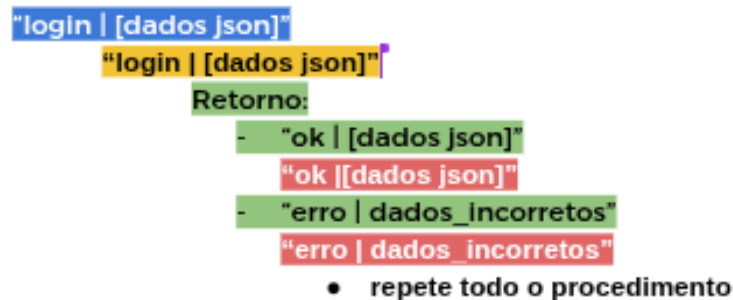


Figura 6: Troca de mensagens para login

4.4.2. Representação externa de dados

Para que a comunicação e transferência de dados fosse efetiva, foi utilizado a forma de conversão das classes para dicionário e json, de modo que em uma classe fosse possível chamar funções que retornam um dicionário convertido para json e uma função que recebe um dicionário ou um json e cria a classe. Essa conversão é utilizada, principalmente, pelo cliente e servidor de banco de dados.

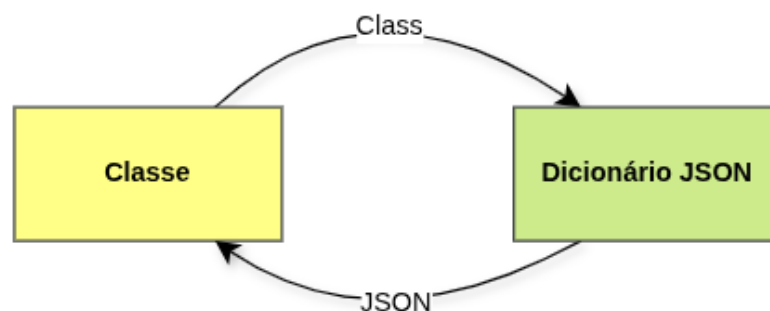


Figura 7: Conversão de representação de dados

4.4.3. Envio e recebimento :

O envio de mensagens ocorre pelo envio inicial do tamanho da mensagem em bytes e depois a mensagem em si, da mesma forma, o recebimento ocorre pelo recebimento inicial de um tamanho e depois da mensagem em si.

O envio de imagem é composto pelo envio do tamanho da imagem em bytes e a imagem em si também em bytes, assim como o envio de mensagens. Porém, o recebimento de imagens é um pouco diferente, visto que imagens são sempre muito grandes e devem ser lidas de modo a juntar todos os bytes para que a imagem seja formada.

Dessa forma, o receptor recebe inicialmente o tamanho, mas a imagem em si é recebida em partes, visto que é necessário especificar o tamanho que está sendo lido, o receptor lê do buffer enquanto o tamanho recebido inicialmente não é alcançado e analisa se a tamanho que está faltando é menor que o buffer que está sendo lido, para que não leia além do tamanho especificado. Após a leitura completa, esses dados são escritos em um caminho do computador para que a imagem seja montada.

5. Resultados :

A aplicação cliente estava de fato pronta para ser utilizada sozinha, enquanto os servidores não puderam ser terminados. As coisas que acontecem como o planejado são: Cadastro de Usuário; Login de Usuário; Criação, Visualização e Atualização de Endereços; Criação de Lojas e Criação de Produtos, visualização de loja, minhas lojas, produtos e anúncios; Atualização de Loja, Produto e Anúncio; Criação de Pedido, mas não sua visualização.

Apesar da entrega ter sido realizada com atraso, ainda sim faltou atender as outras requisições de pedidos (respectivamente, visualizar, confirmar e cancelar pedidos), o que terá que ficar para a próxima implementação, além de alguns ajustes na cascata de exclusões.

6. Dificuldades :

A maior parte dos integrantes do grupo já se havia desfamiliarizado com Python, sendo a primeira barreira a ser vencida os pequenos pormenores e especificidades da linguagem. Outro grande obstáculo foi manter a integridade das interações: contando com uma extensa variedade de mensagens, muitas vezes duas partes da implementação acabaram se desentendendo.

O sistema mostrou precisar de maior coordenação no desenvolvimento dos múltiplos processos. O ponto acima, sobre a desfamiliarização com Python, influenciou a dificuldade de fazer um sistema desacoplado o suficiente para que progressos feitos não fossem desfeitos frente a um problema identificado apenas posteriormente.

Admitidamente, a padronização das mensagens e as ordens de envio não foi elaborada com o rigor e a qualidade necessários para garantir um desenvolvimento fluido das trocas de mensagens. Assim, as operações que envolvem lidar com fluxo de imagens foram as que mais emperraram o desenvolvimento do programa.

7. Conclusão :

Embora barrados por inúmeros obstáculos e contratempos, o grupo conseguiu compreender bem o uso de sockets para comunicação entre pontos numa rede, tendo conseguido enviar e receber imagens. Entretanto, o trabalho se mostrou uma tarefa longe de ser trivial. O desenvolvimento de um sistema distribuído revelou-se desafiador e extenso.

8. Referências :

[1] DATACAMP. *A complete guide to socket programming in Python*. Disponível em: <https://www.datacamp.com/tutorial/a-complete-guide-to-socket-programming-in-python>. Acesso em: 20 maio 2025.

[2] PYTHON SOFTWARE FOUNDATION. *Socket Programming HOWTO*. Disponível em: <https://docs.python.org/3/howto/sockets.html>. Acesso em: 19 maio 2025.

[3] PYTHON SOFTWARE FOUNDATION. *threading — Thread-based parallelism*. Disponível em: <https://docs.python.org/3/library/threading.html>. Acesso em: 19 maio 2025.

[4] PYTHON SOFTWARE FOUNDATION. *queue — A synchronized queue class*. Disponível em: <https://docs.python.org/3/library/queue.html>. Acesso em: 20 maio 2025.

[5] GEEKSFORGEES. *Queue in Python*. Disponível em: <https://www.geeksforgeeks.org/queue-in-python/>. Acesso em: 20 maio 2025.

[6] COULOURIS, George; DOLLIMORE, Jean; KINDBERG, Tim; BLAIR, Gordon. *Sistemas Distribuídos: conceitos e projeto*. 5. ed. Porto Alegre: Bookman, 2013.